IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS

DISCRETE MATHEMATICS

Tuesday, 13 May 2025, 2:30 PM

Time allowed: 2:00 hours

There are THREE questions on this paper.

Answer ALL questions.

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : D. Goodman
Second Marker(s) : Sen Wang

© Imperial College London

DISCRETE MATHEMATICS

1. This question carries 30% of the marks.

   a) For each of the following statements, state whether it is true or false and provide a supporting proof.

      i) $2n+1 = O(n)$. [3]

      ii) $n+(-1)^n n = \Theta(n)$. [3]

      iii) $n^2 = \omega(n(n+1))$ [3]

   b) Give a tight bound for each of the following recurrence relations, or explain why it is not possible to do so.

      i) $T(n) = 9T(n/3)+n$. [2]

      ii) $T(n) = 4T(n/4)+n^3$. [2]

      iii) $T(n) = 8T(n/4)+O(n^2)$. [2]

Master Theorem. If $T(n)$ satisfies

$$T(n) = a\,T(\,n/b\,) \;+\; O(n^d)$$

for some $a > 0$, $b > 1$ and $d \geq 0$, then

$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \\ O(n^d \log n) & \text{if } d = \log_b a \\ O(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

2. This question carries 20% of the marks.

The quicksort algorithm is defined as follows:

```
function QUICKSORT(A)
    q ← PARTITION(A)
    QUICKSORT(A[1 ··· q − 1])
    QUICKSORT(A[q + 1 ··· n])
end function
function PARTITION(A)
    p ← A[n]
    i ← 0
    for j = 1 to n − 1 do
        if A[j] ≤ p then
            i ← i + 1
            swap A[i] and A[j]
        end if
    end for
    swap A[n] and A[i + 1]
    return i + 1
end function
```

It works by choosing a "pivot" element p (the last element of the array in the implementation above), and iterating through the list to ensure that all values of A less than p are to the left of p and all values greater than p are to the right. The array is then split into two parts, and the algorithm is called recursively on each part.

a) Write down a tight bound using Θ notation for the time complexity of the PARTITION function. [1]

b) Write down a recurrence relation for $T(n)$, the time complexity of the quicksort algorithm, in terms of the location q of the pivot element. [1]

c) What is the worst case time complexity of the quicksort algorithm? [2]

d) What is the best case time complexity of the quicksort algorithm? [2]

e) In this part we will compute the average case time complexity. Let $C(n)$ be the average number of comparisons made by the quicksort algorithm on an array of length n.

i) Write down a recurrence relation for $C(n)$ in terms of the location q of the pivot element. [1]

ii) Assume each location of the pivot element is equally likely. Show that

$$C(n) = n - 1 + \frac{2}{n}\sum_{i=0}^{n-1} C(i).$$

[1]

iii) By calculating $nC(n) - (n-1)C(n-1)$, show that

$$\frac{C(n)}{n+1} \leq \frac{C(n-1)}{n} + \frac{2}{n+1}.$$

[1]

iv) Show that $C(n) = O(n \log n)$.

Hint: use the fact that $\sum_{i=1}^{n-1} \frac{1}{i} \approx \log n$. [1]

3. This question carries 50% of the marks.

Consider the following pseudocode designed to run on a single processor:

```
function CACHED
    for i = 1 to m do
        x ← X_i
        Y_i ← f(x)
    end for
    for j = 1 to n do
        i ← h(j) and 1 ≤ i ≤ m
        y ← Y_i
        Z_j ← g(y)
    end for
end function
```

The computation of the array Z is broken down into two parts: first an intermediate array Y of length m is computed, and then this array is re-used in the computation of the array Z of length n. Assume that the time taken to do one read and one write to memory is t_m, that each computation f or g takes time t_c, and that all other operations take negligible time. Assume also that $m = \alpha n$ for some constant $0 < \alpha < 1/2$.

a) Compute the time taken to run this pseudocode, assuming all other operations take negligible time. [4]

b) Write a parallel version of this algorithm that still computes the intermediate array Y. You may use the "parallel for" and "sync" keywords. [2]

c) Assume that computation can be done in parallel, but that reads and writes to the arrays X, Y and Z in global memory are queued in series. In other words, if thread 1 and thread 2 both read from global memory at the same time, one thread will have to wait for the other thread's read to complete. Compute the work T_1 and span T_∞ of your parallel algorithm, ignoring the time taken to launch threads. [4]

d) Write a parallel version of this algorithm (named RECOMPUTE) that computes the array Z directly, without computing Y. [4]

e) Compute the work T_1 and span T_∞ of your parallel algorithm in the previous part. [4]

f) On a single processor system, which algorithm should you use if $t_m = t_c$? [1]

g) On a system with an infinite number of processors, which algorithm should you use? [1]

h) Consider a system with p processors. Assume threads are scheduled with a greedy scheduler and that this scheduler has worst case performance $T_p = T_1/p + T_\infty$. For which values of p should you use the CACHED and RECOMPUTE algorithms when $t_m = t_c$? [5]